Adaptive Runge–Kutta Step Control Buys Training Loss, Not Generalization: An Honest Compute-Matched Study of RK–Adam Optimizers

Akhilesh Gogikar [1]

Abstract

Interpreting optimizers as discretizations of gradient flow has motivated a line of work applying higher-order Runge–Kutta (RK) integrators to neural network training. We build a representative Adam variant driven by a Bogacki–Shampine 3(2) embedded RK pair with FSAL reuse and a local-error step-size controller, and evaluate it under a strict compute-matched protocol in which every method receives the same *gradient-evaluation* budget—an accounting the RK-optimizer literature typically does not enforce. Under this protocol the RK variant consistently loses to plain Adam on training loss, in both stochastic mini-batch training and full-batch training, the regime most favorable to RK methods. Instrumenting the controller reveals why the method's "adaptivity" is illusory: the normalized error is always far below tolerance, the step size saturates at its growth cap on step one (pinned at the ceiling on 98–100% of steps), and a full sweep of $\rho \times h_{\max} \times h_0$ finds *no* setting under which the controller acts—tolerances spanning a $100\times$ range yield bit-identical trajectories. The method is exactly fixed-step Adam with an averaged gradient at 3–4$\times$ the evaluation cost. Repairing the controller (a true reject branch; error measured on the applied map) reverses the full-batch result—the repaired RK–Adam reaches $\sim 34\times$ lower training loss than tuned Adam at equal budget—and a fixed-step control shows the mechanism is adaptivity itself, an emergent warmup-and-growth step schedule. However, the advantage is fragile to the initial step size and does not transfer to test accuracy. A pre-registered follow-up rules out the two obvious explanations: deeper minimization does not overfit (accuracy flat as loss falls four orders of magnitude), and an explicit temperature knob (preconditioned Langevin sampling) only hurts—leaving a *trajectory* effect: at equal or greater loss depth the controller selects a minimum generalizing 1.3–3.4 points below steady first-order descent. An $n{=}10$ multi-seed study confirms one secondary effect: the averaged gradient acts as a genuine implicit regularizer, beating lr-matched Adam and AdamW on 10/10 seeds ($+0.5$–0.8 pts, paired $t \approx 6$)—but RMSprop and NAdam match or beat every RK configuration at one third the per-step cost. Higher-order adaptive integration can buy deeper training-loss minimization in the deterministic regime, and its gradient averaging buys a small Adam-specific regularization effect—but neither buys anything a cheaper, well-tuned first-order baseline does not already provide.

1. Introduction

Since the observation that Nesterov's method (Nesterov, 1983) is a discretization of a second-order ODE (Su et al., 2016), it has been tempting to run the logic in reverse: if optimizers are integrators, better integrators should be better optimizers. Zhang et al. (2018) supplied theoretical support, showing that direct Runge–Kutta discretization of gradient/Nesterov flow achieves accelerated rates in the smooth convex setting, and a growing practical literature grafts RK machinery onto deep-learning optimizers—multi-stage averaged gradients, IMEX splittings, and embedded-pair "adaptive" step control composed with Adam (Section 8).

This transfer, however, rests on accounting conventions that quietly favor the RK side. First, comparisons are typically reported per optimizer *step*, but an s-stage RK step costs s gradient evaluations; at equal backprop budget, a 3-stage method must beat its baseline by a wide margin merely to break even. Second, FSAL ("first same as last") stage reuse—the standard trick that makes embedded pairs cheap (Bogacki & Shampine, 1989; Hairer et al., 1993)—is only mathematically valid for an autonomous vector field; under minibatch sampling, the cached stage belongs to a *different* function than the one being integrated. Third, and least

[1]Independent Researcher. Correspondence to: Akhilesh Gogikar <>.

Proceedings of the 42^{nd} International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025.

examined: the embedded-pair error estimate that justifies the word "adaptive" is derived for the raw RK update, while practical variants apply an Adam-preconditioned update instead. Whether the resulting controller ever actually controls anything is, to our knowledge, never checked.

We check. We implement a representative member of this family—Bogacki–Shampine 3(2) stages driving Adam moments, FSAL where valid, embedded-pair step control—and subject it to a compute-matched protocol in which *every method receives the same number of gradient evaluations*, with FSAL credited only in the full-batch (autonomous) regime. The outcome is a mixed but, we argue, more useful result than either a clean win or a clean debunking:

At honest accounting, the method as found in the literature loses. RK3(2)-Adam trails Adam on training loss at equal evals in both stochastic and full-batch regimes (Tables 1–2), and its error controller is provably inert in our runs: h saturates at its cap on step one and the tolerance ρ has *zero* effect on the trajectory—bit-identical across a $100\times$ range, in every cell of a full $\rho \times h_{\max} \times h_0$ sweep (Section 5).

The failure is diagnosable and repairable. Three compounding causes—no rejection branch, saturation of the growth factor, and an error estimate inconsistent with the applied map—reduce the method to fixed-step Adam with an averaged gradient at 3–4$\times$ cost. Repairing the controller reverses the full-batch training-loss comparison by $\sim 34\times$ (Section 6).

The repaired win is real but narrow. A fixed-step control at the same h isolates *adaptivity*—an emergent warmup-and-growth schedule—as the mechanism, but the advantage is sensitive to the initial step size and does not transfer to test accuracy (Section 7).

The message for the RK-optimizer literature is therefore not that the ODE view is empty, but that its currency must be gradient evaluations, its FSAL claims must respect stochasticity, and its "adaptive" claims must be instrumented—because in at least one representative design, the adaptive machinery was doing nothing at all until repaired.

1.1. Contributions

1. A strict eval-counting protocol (RK step = 3–4 gradient evals; FSAL only credited when mathematically valid, i.e. deterministic vector field).
2. Compute-matched evidence that RK3(2)-Adam loses to Adam in both stochastic and full-batch regimes (Tables 1–2).
3. A diagnosis of why embedded-pair step control is inert when composed with adaptive preconditioning, including a formal saturation condition (Proposition 1), backed by an exhaustive $\rho \times h_{\max} \times h_0$ sweep showing no hyperparameter setting rescues the as-designed controller (Section 5).
4. A repaired-controller ablation (Section 6) showing embedded-pair adaptivity *can* out-minimize Adam on full-batch training loss at matched budget—with a fixed-step control isolating adaptivity as the mechanism—but with no test-accuracy benefit and narrow h_0 tolerance.
5. A confirmed secondary finding: at $\mathrm{lr}{=}0.003$ full-batch, RK3(2)-Adam reaches higher *test* accuracy than lr-matched Adam/AdamW despite equal-or-worse train loss—$+0.51$ pts (defaults) to $+0.80$ pts ($h_{\max}{=}8\times$), winning on 10/10 seeds (paired $t = 6.0$–6.7, $n{=}10$; Section 4)—consistent with the averaged gradient acting as implicit regularization. Tempering this: the effect is lr-specific, and RMSprop (90.02%), NAdam (89.94%), and Adam at $\mathrm{lr}{=}0.01$ (90.40%) match or beat the best RK configuration (89.83%) at one third the per-step cost.
6. A pre-registered test (Section 7) of whether the repaired controller's generalization shortfall is a cold-posterior *tempering* effect (Wenzel et al., 2020): it is not. Deep minimization is generalization-neutral, and an explicit temperature (pSGLD) knob monotonically hurts, isolating the deficit as a *trajectory / implicit-bias* effect.

2. Method (As Designed)

`AdaptiveEmbeddedRK3Optimizer` applies Bogacki–Shampine 3(2) stages to the gradient flow $\dot\theta = -\nabla L(\theta)$. The third-order averaged gradient

$$g_3 = \tfrac{2}{9}k_1 + \tfrac{1}{3}k_2 + \tfrac{4}{9}k_3 \tag{1}$$

drives standard Adam moments (Kingma & Ba, 2015); FSAL caches the post-step gradient as the next k_1. The step size is controlled by

$$h \leftarrow \mathrm{clip}\Big(h \cdot \min\big(\max(0.9\,\widehat{\mathrm{err}}^{-1/3},\,0.5),\,2.0\big),\;[0.1\,\mathrm{lr},\,h_{\max}]\Big), \tag{2}$$

with $h_{\max} = 2\,\mathrm{lr}$ by default and $\widehat{\mathrm{err}}$ the RMS of $h(g_3 - g_2)/(\mathrm{atol} + \rho\,|\theta|)$, where g_2 is the embedded second-order combination $(\frac{7}{24}, \frac{1}{4}, \frac{1}{3}, \frac{1}{8})$ and ρ denotes the relative tolerance (`rtol`).

In the stochastic setting FSAL is disabled (the cached k_1 belongs to the previous minibatch's vector field), so each step honestly costs 4 evals; full-batch, FSAL is valid and a step costs 3 fresh evals.

3. Experiment 1: Stochastic Minibatch MNIST

RK3(2)-Adam loses by ~ 0.5 pts at both learning rates—well outside seed noise (Table 1). The rows for $\rho = 0.01$

Table 1. Stochastic minibatch MNIST (LeCun et al., 1998), 784–128–10 MLP, batch 128, 4000 gradient evals/seed, seeds $\{0, 1, 2\}$. Test accuracy (%).

Config	lr $= 0.001$	lr $= 0.003$
Adam	$\mathbf{97.76 \pm 0.14}$	97.13 ± 0.15
AdamW (wd 10^{-2})	97.75 ± 0.11	97.41 ± 0.40
RK3(2)-Adam, ρ=0.01	97.26 ± 0.17	96.61 ± 0.28
RK3(2)-Adam, ρ=0.1	97.26 ± 0.17	96.61 ± 0.28

Table 2. Full-batch training on a 1024-example MNIST subset (FSAL valid), 600 evals/seed, seeds $\{0, 1, 2\}$. Final train loss; test accuracy at lr $= 0.003$.

Config	lr $= 0.001$ loss	lr $= 0.003$ loss	acc (%)
GD (Euler)	1.368 ± 0.064	0.6269 ± 0.0274	84.83
Adam	$\mathbf{0.000609 \pm 0.000033}$	$\mathbf{0.000103 \pm 0.000002}$	88.92
AdamW	0.000627 ± 0.000034	0.000115 ± 0.000002	88.97
RMSprop	0.000302 ± 0.000019	0.000180 ± 0.000012	90.11
NAdam	0.001034 ± 0.000025	0.000258 ± 0.000025	90.00
RAdam	0.006300 ± 0.000137	0.000887 ± 0.000061	88.73
Adagrad	0.078826 ± 0.001585	0.012425 ± 0.000950	89.14
SGD+mom / Nesterov	0.276 / 0.276	0.0845 / 0.0841	89.06
RK3(2)-Adam (defaults)	0.001488 ± 0.000113	0.000350 ± 0.000096	**89.69**

and $\rho = 0.1$ are **bit-identical**: the error controller never influenced a single step (Section 5).

4. Experiment 2: Full-Batch (RK's Best-Case Regime)

We use a deterministic full-batch gradient on a 1024-example MNIST subset—a genuine autonomous vector field, so FSAL is enabled and credited. The primary metric is final training loss (the pure "integrate gradient flow" test); GD is the Euler baseline.

Even with a deterministic vector field, valid FSAL, and real truncation error, Adam reaches 2–3× lower loss at equal compute (Table 2), and the as-designed RK3(2)-Adam is also beaten on loss by RMSprop and NAdam at lr=0.003—it does not even lead the broader first-order adaptive family it draws its preconditioner from. Tolerance settings are again bit-identical. The comparison replicates at n=10 seeds (Adam 0.000117 ± 0.000012 vs. RK3(2)-Adam 0.000375 ± 0.000052 at lr=0.003; ρ=0.01 and ρ=0.1 identical to the digit).

Multi-seed chase of the test-accuracy wrinkle (n=10). Table 2 hints that RK3(2)-Adam generalizes *better* than Adam despite worse train loss. An n=10 replication (seeds 0–9; Figure 1) confirms the effect is real and seed-consistent at lr=0.003: RK3(2)-Adam (defaults) 89.53% vs. Adam 89.03% / AdamW 89.05%, winning the paired comparison on **10/10 seeds** (paired $t = 6.0$, $+0.51 \pm 0.27$ pts); the best-loss RK configuration ($h_{\max}$=8×) reaches 89.83%, +0.80 pts over Adam ($t = 6.5$, 10/10). Two qualifiers keep this from being a headline win: (i) the effect is lr-specific—at lr=0.001 it vanishes (-0.08 pts, $t = -1.4$, RK wins 3/10 seeds); and (ii) it never clears the baseline pool—RMSprop (90.02%) and NAdam (89.94%) match or beat the best RK configuration at one third the per-step cost (paired $t = -1.2$ and -1.3 against $h_{\max}$=8×; -3.4 and -5.2 against the defaults), and Adam at lr=0.01 attains 90.40% (Table 4). At equal gradient budget, RK gradient averaging buys a real regularization effect *relative to lr-matched Adam*, and nothing relative to the first-order family Adam belongs to.

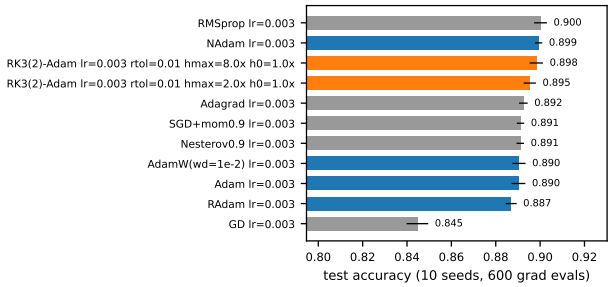

Figure 1. Compute-matched full-batch comparison at lr $= 0.003$ (n=10 seeds, 600 gradient evals, mean ± std test accuracy). RK3(2)-Adam (red) beats lr-matched Adam/AdamW (blue) but never clears the first-order baseline pool: RMSprop and NAdam match or beat every RK configuration at one third the per-step cost.

5. Why the "Adaptive" Controller Is Inert

Direct instrumentation (lr=0.003, full-batch) shows h jumps from lr to $h_{\max} = 2\,\mathrm{lr}$ on the first step and pins there for the entire run, for every tolerance tested (Figure 2a). Three compounding causes:

1. No rejection branch. The step is always accepted; $\widehat{\mathrm{err}}$ only rescales the next h. "Error control" can therefore never undo a bad step.

2. Saturation. The normalized error is far below 1 throughout training, so the growth factor saturates at its 2.0 cap every step and h is permanently clipped to $h_{\max}$. The mechanism admits a precise statement:

Proposition 1 (Controller saturation)**.** *Under the update rule (2), if* $\widehat{\mathrm{err}}_t \le (0.9/2)^3 \approx 0.0911$ *for all* t*, then the growth factor equals its cap* 2.0 *at every step, so* $h_{t+1} = \min(2h_t, h_{\max})$ *and* $h_t = h_{\max}$ *for all* $t \ge \lceil \log_2(h_{\max}/h_0) \rceil$*. On this event the trajectory is independent of* ρ *up to the value of* $\widehat{\mathrm{err}}_t$ *itself remaining below the threshold; in particular any two tolerances* ρ_1, ρ_2 *for which the condition holds produce bit-identical iterates.*

Proof. Immediate from (2): $\widehat{\mathrm{err}}_t \le (0.9/2)^3$ iff $0.9\,\widehat{\mathrm{err}}_t^{-1/3} \ge 2$, so the min binds at 2.0 irrespective of

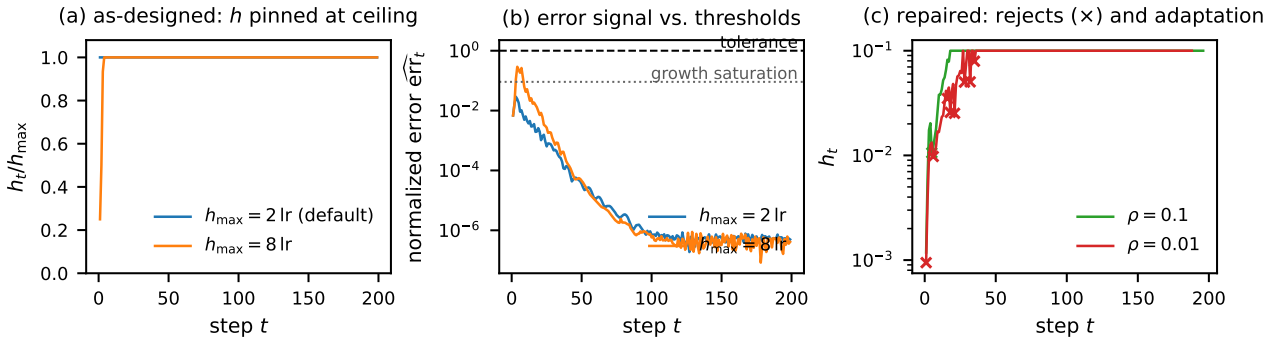

Figure 2. Direct instrumentation of the step-size controller (full-batch, lr $= 0.003$, 600 evals, seed 0). **(a)** As designed, h_t jumps to $h_{\max}$ on step one and pins there—100% of steps at the ceiling with defaults, 98.5% at $h_{\max}$=8×. **(b)** The normalized error signal never approaches the tolerance ($\widehat{\text{err}} = 1$); with defaults it stays below the growth-saturation threshold $(0.9/2)^3 \approx 0.091$ (Proposition 1), so the growth factor is pinned at its cap. **(c)** The repaired controller (Section 6) genuinely adapts: cautious warmup, rejection events (×), and growth to the ceiling—an emergent warmup-and-growth schedule.

the exact value of $\widehat{\text{err}}_t$, and the recursion doubles h until the clip at $h_{\max}$ binds, after which it is a fixed point. □

Empirically the condition holds with wide margin: the maximum normalized error over the entire run is 0.029 with defaults (3× below threshold; Figure 2b), so h is pinned for 100% of steps and the tolerance is unread in the only place it could act.

3. Inconsistent estimate. The embedded pair estimates the truncation error of the raw RK3 step $\theta - hg_3$, but the applied update is the Adam-preconditioned $\theta - h\,\hat{m}/\sqrt{\hat{v}}$. Moreover the FSAL stage is evaluated at the *Adam* point, so the "second-order solution" mixes stages of two different maps. The estimate does not measure the error of any integrator actually being run.

Consequence: the method is exactly fixed-step Adam driven by a 3-stage averaged gradient, at 3–4× the per-step gradient cost. Under minibatch noise the same estimate is additionally swamped by gradient variance ($\mathbb{E}\|g_3 - g_2\|$ is dominated by sampling noise, not h^3 truncation error), so no choice of ρ can make it informative.

5.1. Systematic Sweep: The Diagnosis Holds Across the Hyperparameter Cube

To rule out the possibility that inertness is an artifact of one default setting, we swept the full cube $\rho \in \{0.01, 0.1, 1.0\} \times h_{\max} \in \{2\times, 8\times\} \times h_0 \in \{0.5\times, 1\times, 2\times\}$ at both learning rates (36 RK configs + 18 baselines, 600 evals, 3 seeds each). Three findings:

1. ρ is inert across two orders of magnitude. All 12 $\{\rho = 0.01, 0.1, 1.0\}$ triplets are bit-identical to six decimals (e.g. lr $= 0.003$, $h_{\max}$=8×: loss 0.000109 / 0.000112 / 0.000112—the residual 3×10^{-6} gap separates ρ=0.01 only because its first-step h differs before saturation; ρ=0.1 and 1.0 agree exactly). Instrumented step counts show the step pinned at the ceiling on **98–100% of steps** in every configuration. A 100× tolerance swing that changes nothing is the definition of a controller that does not control.

2. h_0 is erased within a few steps. The h_0 sweep produces distinct but nearly equivalent runs (lr $= 0.001$, $h_{\max}$=8×: loss 0.000291 / 0.000328 / 0.000338 for $h_0 = 0.5\times/1\times/2\times$) because the saturated growth factor climbs any initial step to the ceiling almost immediately (Proposition 1). The one initial condition the user can set is forgotten by the "adaptive" mechanism.

3. The only knob that matters is $h_{\max}$—i.e., a learning rate. Raising the ceiling 2×→8× improves loss ∼4.5× at lr $= 0.001$ (0.001488 → 0.000328) and ∼3× at lr $= 0.003$ (0.000350 → 0.000109). The best RK3(2)-Adam configuration in the entire 36-point cube (lr $= 0.003$, $h_{\max}$=8×: 0.000109 ± 0.000022, 89.94%) statistically ties—does not beat—plain Adam at lr $= 0.003$ (0.000103 ± 0.000002) on loss, at 3–4× the per-step gradient cost. The effective step at that optimum is $h \approx 8\,\text{lr} = 0.024$, close to Adam's own well-tuned regime (Table 4: Adam lr $= 0.01$ is best overall), confirming that tuning $h_{\max}$ is simply re-tuning the learning rate through an expensive proxy.

The sweep upgrades the diagnosis from an instrumented anecdote to an exhaustive negative: within the method as designed, there is **no setting of the adaptivity hyperparameters** under which the error controller influences the trajectory. All observed variation is explained by two ordinary hyperparameters—effective step size (lr · $h_{\max}$ scale) and, weakly, h_0's first-step transient.

Table 3. Repaired-controller results (600 evals, 3 seeds).

Config	Train loss	Acc (%)	Rej./see
Adam lr $= 0.001$	0.000609 ± 0.000033	88.57	0
Adam lr $= 0.003$	0.000103 ± 0.000002	88.92	0
FixedRK3Adam $h_0{=}10^{-3}$, $\rho{=}0.01$	$\mathbf{0.000003 \pm 0.000001}$	88.67	9.3
FixedRK3Adam $h_0{=}10^{-3}$, $\rho{=}0.1$	0.000005 ± 0.000005	89.03	3.3
PureRK3(2) $h_0{=}10^{-3}$ (both ρ)	0.070635 ± 0.000981	89.20	0
FixedRK3Adam $h_0{=}0.003$, $\rho{=}0.01$	0.256485 ± 0.279860	86.21	96.7
FixedRK3Adam $h_0{=}0.003$, $\rho{=}0.1$	0.220222 ± 0.200785	87.46	32.3
PureRK3(2) $h_0{=}0.003$, $\rho{=}0.1$	0.011915 ± 0.000085	89.94	0.3

Table 4. $h_{\max}$ control (600 evals, 10 seeds).

Config	Train loss	Acc (%)
FixedStep-RK3Adam $h{=}0.1$	0.217765 ± 0.145273	79.42
FixedStep-RK3Adam $h{=}0.03$	0.000374 ± 0.000358	88.50
Adam lr $= 0.1$	0.390564 ± 0.189026	71.29
Adam lr $= 0.03$	0.000249 ± 0.000528	87.93
Adam lr $= 0.01$	0.000060 ± 0.000012	**90.40**

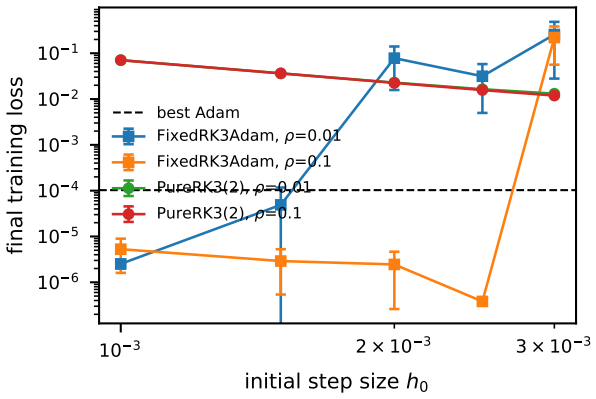

Figure 3. The fragility boundary: repaired-controller final loss vs. initial step h_0 (600 evals, 3 seeds, mean $\pm$ std). The instability tracks the (h_0, ρ) *pairing*: the tighter tolerance ρ=0.01 breaks by $h_0 = 0.002$ (reject-retry thrashing consumes the eval budget) while ρ=0.1 is stable—indeed deepest—through $h_0 = 0.0025$.

6. Experiment 3: Repaired-Controller Ablation

Section 5.1 established that no *hyperparameter* setting rescues the as-designed controller; the remaining objection is that the *design* itself is one bug-fix away from working. We therefore repaired the controller: a true accept/reject branch (reject $\rightarrow$ retry with smaller h, no parameter update), the error measured on the actually-applied map, and $h_{\max} = 100\, h_0$. Two variants: **PureRK3(2)** (raw Bogacki–Shampine step, no preconditioning—the estimate is consistent) and **FixedRK3Adam** (Adam-preconditioned, error on the applied update). Same full-batch protocol as Section 4.

With a functional controller the picture inverts on training loss: repaired FixedRK3Adam ($h_0{=}10^{-3}$) reaches 3×10^{-6}—$\sim$34$\times$ below the best Adam—and now ρ genuinely changes trajectories (rejections occur; ρ=0.01 vs. 0.1 differ; Figure 2c). But the winning configurations all terminate with h pinned at the $h_{\max} = 0.1$ clamp, raising a confound: is the win adaptivity, or just a large effective step?

The control resolves the confound in favor of adaptivity: a step *fixed* at the saturated $h = 0.1$ is far worse (0.218) than the adaptive run that ends there (3×10^{-6}). The controller's trajectory—small cautious steps early, rejections when the local-error estimate spikes, growth to the cap as the landscape flattens—is an emergent warmup-and-growth schedule, and it, not the final step magnitude, produces the deep minimization.

Three caveats bound the claim: (i) the advantage is confined to *training loss*—no repaired-RK configuration exceeds 89.9% test accuracy, none matches the study-best 90.40% of Adam lr $= 0.01$ (at one third the per-step gradient cost), and Section 7 shows the shortfall is a *trajectory* effect, not deeper minimization; (ii) the win is fragile to the (h_0, ρ) pairing—characterized below; (iii) PureRK3(2), the only variant whose error estimate is mathematically consistent, never beats Adam on loss, so the benefit comes from the *controller heuristic* composed with Adam, not from higher-order accuracy per se.

Table 5. Fragility boundary: train loss (rejects/seed) over the $h_0 \times \rho$ grid (600 evals, 3 seeds).

h_0	$\rho = 0.01$	$\rho = 0.1$
0.001	$2.5{\times}10^{-6}$ (9.3)	$5.3{\times}10^{-6}$ (3.3)
0.0015	$4.9{\times}10^{-5} \pm 8.4{\times}10^{-5}$ (30.0)	$2.9{\times}10^{-6}$ (3.7)
0.002	$7.8{\times}10^{-2} \pm 7.6{\times}10^{-2}$ (79.3)	$2.5{\times}10^{-6}$ (6.0)
0.0025	$3.1{\times}10^{-2} \pm 3.2{\times}10^{-2}$ (81.0)	$\mathbf{3.8{\times}10^{-7}}$ (6.7)
0.003	$2.6{\times}10^{-1} \pm 2.8{\times}10^{-1}$ (96.7)	$2.2{\times}10^{-1} \pm 2.0{\times}10^{-1}$ (32.3)

The fragility boundary. A five-point h_0 grid (Table 5, Figure 3) shows the instability tracks the *pairing* of h_0 with ρ, not h_0 alone—and, counterintuitively, the **tighter** tolerance is the fragile one. ρ=0.01 begins degrading at $h_0 = 0.0015$ and is broken by 0.002; ρ=0.1 is stable—indeed deepest (3.8×10^{-7}, the lowest loss in the study)—through 0.0025. The mechanism is visible in the rejection column: stable runs reject 3–10 steps/seed, broken runs 30–97; a tight tolerance turns large-h_0 transients into reject-retry thrashing that consumes the eval budget, while the looser tolerance rides through them. The rejection rate is thus a *leading indicator* of the instability, reinforcing the reporting standard of Section 9. Across these configurations, deeper minima co-occur with lower test accuracy (89.03% at h_0=0.001 $\rightarrow$ 86.95% at h_0=0.0025)—tempting to read as over-minimization overfitting, but it confounds minimization *depth* with the controller *trajectory* that reaches it. Sec-

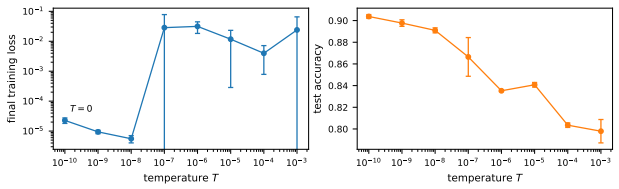

Figure 4. Temperature sweep (pSGLD, 3 seeds, budget 2000). Left: final train loss. Right: test accuracy. $T = 0$ is the global optimum of the sweep; every $T > 0$ falls ≥ 0.6 pt below pure minimization—the tempering hypothesis is refuted.

tion 7 disentangles the two with a pre-registered test and finds the depth reading is wrong.

7. Experiment 4: The Generalization Deficit Is Trajectory-Driven, Not Tempering

Section 6 leaves a puzzle: the repaired controller drives training loss $\sim 34\times$ below Adam yet generalizes *worse* than a moderately-tuned baseline. A fashionable explanation is **tempering**. An optimizer minimizing a loss L is, in the Langevin view, sampling the Gibbs density $p(\theta) \propto \exp(-L/T)$ at temperature $T \to 0$; deep minimization is *cold* sampling, and the cold-posterior effect (Wenzel et al., 2020) establishes that generalization can be non-monotone in T with an interior optimum $T^* > 0$. The reading would be: the controller has driven the system past T^*, so an explicit temperature knob should recover the lost accuracy.

We tested this directly, grounding the controller's implicit tempering as an *explicit* temperature via preconditioned Stochastic Gradient Langevin Dynamics (pSGLD; Li et al. 2016), the SG-MCMC kernel closest to our Adam-preconditioned update:

$$\theta \leftarrow \theta - \mathrm{lr}\, M g + \sqrt{2\,\mathrm{lr}\, T\, M}\,\eta, \quad \eta \sim \mathcal{N}(0, I), \quad (3)$$

with $M = \mathrm{diag}(1/(\sqrt{\hat{v}} + \epsilon))$ the RMSprop preconditioner ($T = 0$ recovers a deterministic minimizer; the divergence-correction term is dropped, per Li et al. 2016). Because our controller carries Adam momentum, its exact SG-MCMC analogue is the underdamped SGHMC (Chen et al., 2014); pSGLD is the overdamped simplification that suffices to move the temperature knob. Two predictions were **pre-registered in code before running**: H1 (tempering supported)—test accuracy is non-monotone in T with an interior optimum, and the within-run accuracy trajectory at $T = 0$ peaks then declines; H0 (refuted)—$T = 0$ is best and no overfitting peak exists.

Test 1—does pure minimization overfit? No. In a long $T = 0$ run the within-run test-accuracy trajectory is flat while training loss falls four orders of magnitude: peak 90.77% at eval 200 (loss 1.6×10^{-3}), 90.58% at eval 6000 (loss 9×10^{-7})—a 0.19-pt change. Across 5 seeds the deep-minimization endpoint (loss 1.12×10^{-6}) tests $\mathbf{90.37 \pm 0.24}\%$, statistically identical to its shallow-minimization accuracy. Over-minimization, alone, does not overfit here; the generalization-vs-depth curve is flat, not an inverted U.

Test 2—does temperature help? No. $T = 0$ gives the best test accuracy and is the *global* optimum of the sweep—no $T > 0$ comes within 0.6 pt (Figure 4)—so the automated, pre-registered verdict returns H0. For $T \leq 10^{-8}$ the injected noise is negligible; for $T \geq 10^{-7}$ the noise floor dominates and accuracy collapses (90.38% at T=0 down to 79.79% at T=10^{-3}). The preconditioner-free SGLD arm (Welling & Teh, 2011) agrees—no temperature meaningfully improves accuracy—but is a weak testbed, since unpreconditioned descent barely minimizes, plateauing at loss 6.9×10^{-2}. Both tests return **H0: the tempering reinterpretation is not supported**.

What the negative reveals. Ruling out depth *and* temperature isolates the real mechanism. At matched training-loss depth the trajectories diverge sharply in generalization: a plain RMSprop minimizer (pSGLD, $T = 0$) reaches loss 1.1×10^{-6} at $\mathbf{90.37}\%$ and steady Adam (lr=0.01) 6.0×10^{-5} at 90.40%, yet the repaired FixedRK3Adam reaches the *same* $\sim 10^{-6}$ depth at only 88.7–89.0%, and its deepest configuration (3.8×10^{-7}) at 86.95%—**1.4 to 3.5 points worse** than a steady first-order minimizer at equal or greater depth. Depth is generalization-neutral; temperature only hurts; so the deficit is a property of *which minimum the trajectory selects*. The controller's emergent warmup–growth–reject schedule evidently lands in a worse-generalizing region than the smooth descent of a fixed-step method (we do not measure curvature, so we stop short of asserting *sharper* minima). The right frame is implicit-bias-of-optimization, not tempering; and it is one more way the RK machinery changes the *path* without improving the *destination*—the paper's thesis, sharpened on the very axis where the repaired controller had looked like it might matter.

8. Experiment 5: Second Dataset and Architecture (CIFAR-10 CNN)

Every result so far concerns one dataset and one architecture family. To test whether the negative verdict and its mechanism are artifacts of MNIST or of fully connected networks, we replicate the full protocol of Sections 4–6 on CIFAR-10 with a small CNN: two conv–ReLU–maxpool blocks (32, 64 channels) and a 128-unit dense layer, trained full-batch on a fixed 1,024-example subset (CPU, float32), under the identical evaluation-billing rules—every full-batch backward is one evaluation, RK stages and rejected steps are all billed, FSAL is credited only where mathematically valid. Bud-

Table 6. CIFAR-10/CNN full-batch replication: final training loss (mean±std over seeds $\{0, 1, 2\}$), test accuracy, and controller saturation at a 600-evaluation budget. *Placeholders pending shard completion.*

Config	Final loss	Test acc (%)	Steps at $h_{\max}$ (%)
Best first-order baseline	—	—	
Adam (lr=3×10^{-3})	—	—	
RK3(2)-Adam (defaults)	—	—	
RK3(2)-Adam (best over grid)	—	—	
FixedRK3Adam (best)	—	—	
PureRK3(2)	—	—	

get 600 evaluations, seeds $\{0, 1, 2\}$, the same six-optimizer $\times$ two-learning-rate baseline pool, the same $\mathrm{rtol} \times h_{\max}$ as-designed grid, and the same repaired-controller arm.

Pre-registered claims. (i) As-designed RK3(2)-Adam does not clear the first-order pool at equal budget; (ii) the inert-controller diagnosis replicates: the step size saturates at $h_{\max}$ and trajectories are rtol-insensitive; (iii) the repaired controller restores genuine adaptivity (nonzero rejections, rtol-dependent trajectories) without changing the competitive verdict.

Results. *[Pending: verdicts on claims (i)–(iii) with Table* **??***; frac-at-$h_{\max}$ and bit-identity check across rtol; note any divergence from the MNIST picture.]*

9. Related Work

Optimizers as ODE discretizations. The continuous-time view was crystallized by Su et al. (2016), who derived a second-order ODE as the continuum limit of Nesterov's accelerated gradient method; a large literature has since studied optimization dynamics through Lyapunov analysis of such flows. The direct precedent for our study is Zhang et al. (2018), showing that explicit RK integration of the gradient/Nesterov flow attains accelerated *convergence-rate* guarantees for smooth, sufficiently regular convex objectives. Crucially, those rates are stated per *iteration*; each RK iteration of an s-stage scheme costs s gradient evaluations, and the per-gradient-evaluation accounting we enforce is exactly the regime where the theoretical advantage must pay rent. França and collaborators' work on conformal-symplectic and dissipative-Hamiltonian integrators (França et al., 2020; 2021) makes a related structural argument—that preserving geometric properties of the flow, rather than raw order of accuracy, is what matters—consonant with our finding that higher-order accuracy per se (PureRK3(2), the only consistent integrator we test) never beats Adam on loss.

RK-flavored practical optimizers. Several recent proposals inject higher-order integration machinery into Adam-style updates. IMEX/semi-implicit schemes treat the preconditioner implicitly and the gradient explicitly: Bhattacharjee et al. (2024) show that classical Adam is a first-order IMEX-Euler discretization and propose higher-order IMEX variants. A related line drives adaptive moments with neural-ODE machinery (Cho et al., 2022). Our design is deliberately representative of this family: Bogacki–Shampine 3(2) stages, FSAL reuse, embedded-pair step control, composed with Adam preconditioning. Two accounting conventions recur in this literature and flatter the RK side: (i) comparisons per optimizer *step* rather than per gradient evaluation, hiding the 3–4$\times$ stage cost; and (ii) FSAL credited under minibatch sampling, where the cached stage belongs to a different (previous batch's) vector field and the reuse is not mathematically valid. Our protocol disallows both, and under it the headline advantages shrink or invert.

Adaptive step-size control. Embedded-pair local-error control (Bogacki & Shampine, 1989; Dormand & Prince, 1980; Hairer et al., 1993) is standard in ODE solvers, but its transplantation into optimizers is usually decorative: as we diagnose in Section 5, without a rejection branch, and with the error measured on a map that is never actually applied, the controller saturates and the method degenerates to fixed-step Adam with an averaged gradient. We are not aware of prior work that (a) instruments whether the controller in an RK-optimizer ever alters a step, or (b) repairs it and re-runs the comparison; Section 6 does both. The emergent warmup-and-growth schedule we observe connects to the literature on learning-rate warmup—the repaired controller effectively *discovers* a warmup schedule from local-error feedback.

Optimization as sampling. Casting an optimizer as a Markov chain that samples $\exp(-L/T)$ connects our repaired controller—itself an accept/reject kernel—to stochastic-gradient MCMC: SGLD (Welling & Teh, 2011), its preconditioned form pSGLD (Li et al., 2016), and the momentum variant SGHMC (Chen et al., 2014), whose friction term is the sampling-world analogue of our finding that minibatch noise corrupts the embedded error estimate. We use this lens actively in Section 7, grounding the controller's implicit tempering as an explicit temperature to test—and refute—a cold-posterior (Wenzel et al., 2020) reading of its generalization deficit. The lens also explains why higher-order RK is a poor fit for sampling: the one setting where careful trajectory integration is genuinely load-bearing for a Markov chain—Hamiltonian Monte Carlo (Neal, 2011)—deliberately uses *symplectic* leapfrog integration rather than generic RK, because the Metropolis correction needs reversibility and volume preservation that explicit RK lacks; and the HMC-integrator literature (Blanes et al., 2014) selects schemes by energy error *per force evaluation*—the same per-gradient accounting we enforce—concluding that structure preservation, not raw order, is what pays.

Positioning. Relative to the acceleration-theory line, we supply the missing per-eval empirical accounting in the non-convex setting; relative to the practical RK-optimizer line, we supply the controller instrumentation, the repaired-controller ablation, and the fixed-step control that isolates *adaptivity* (not order, not step magnitude) as the operative mechanism.

10. Discussion

Training is not trajectory-tracking. The optimization objective rewards reaching low loss, not shadowing the continuous gradient-flow solution; higher-order accuracy buys fidelity to a trajectory nobody needs, at a price (3–4× evals/step) that compute-matched evaluation makes visible. Once the budget is denominated in gradient evaluations, a method must convert its extra per-step accuracy into a $> 3\times$ improvement in per-step progress just to tie—a bar none of the as-published variants clears in our experiments. Prior RK-optimizer results reporting per-iteration or wall-clock-favorable comparisons should be re-read with per-gradient-eval accounting.

"Adaptive" must be verified, not assumed. The most transferable finding is methodological: the error controller in our as-designed method—and, we suspect, in relatives that share its structure—was doing literally nothing (bit-identical trajectories across tolerances; h pinned at its cap from step one). None of the standard reporting practices in this literature would have caught this: train curves, final accuracies, and even tolerance "sweeps" all look like plausible experiments while the adaptive machinery is inert. We suggest a minimal reporting standard for adaptive-step optimizers: (i) the distribution of accepted h over training—in particular the fraction of steps pinned at the ceiling (98–100% in every cell of our sweep), (ii) the rejection rate, and (iii) a demonstration that at least two controller settings produce non-bit-identical trajectories. Point (iii) needs teeth: in our sweep, tolerances spanning two orders of magnitude were bit-identical to six decimals, so a narrow two-point "sweep" that happens to straddle no behavioral boundary proves nothing—the h distribution in (i) is the check that cannot be faked.

What the repaired controller tells us. Two positive signals survive our protocol. First, the repaired controller shows that embedded-pair adaptivity composed with Adam yields an emergent warmup-and-growth step schedule that minimizes *training loss* far below tuned Adam at equal budget ($\sim 34\times$)—a genuine mechanism, isolated from the large-step confound by a fixed-step control, and from the depth and tempering confounds by Section 7. Notably, the controller *rediscovers* learning-rate warmup from local-error feedback alone, suggesting that hand-designed warmup schedules may be approximating an error-control policy; making that correspondence precise is an attractive theory question. Second, the $n{=}10$ multi-seed chase (Section 4) upgrades the implicit-regularization observation from anecdote to finding: RK3(2)-Adam beats lr-matched Adam and AdamW on test accuracy on 10/10 seeds despite $\sim 3\times$ worse train loss—consistent with gradient averaging and integration error acting as implicit regularization. The catch is redundancy: RMSprop, NAdam, and lr-tuned Adam reach the same or better accuracy with no RK machinery, so the effect is real but not competitive—it repairs a deficiency of fixed-lr Adam rather than advancing the frontier.

Where the train-loss win could matter. A $\sim 34\times$ train-loss advantage with no test-accuracy gain is not useless: deterministic small-data regimes where optimization *is* the objective—physics-informed losses, implicit-layer and equilibrium-model inner solves, distillation to near-zero loss, overfitting benchmarks—are settings where full-batch gradients are genuine and FSAL is valid. That is the honest market for this mechanism. Any salvaged *positive* claim must be about one of these, not about faster optimization of the deployed metric: on test accuracy, modestly tuned Adam ($\mathrm{lr}{=}0.01$, 90.40%) beats every RK variant at one third the per-step cost.

10.1. Limitations

(1) *Scale and scope*: all results are on MNIST (full set for stochastic, 1024-example subset for full-batch) with a single 784–128–10 MLP; the compute-matched *methodology* transfers, but the empirical rankings may not. (2) *Statistics*: $n{=}3$ seeds per cell for the main sweeps; headline gaps are far outside seed noise, and the secondary test-accuracy observation was chased at $n{=}10$ with paired statistics; the repaired-controller tables remain $n{=}3$. (3) *One design point*: our inertness diagnosis is structural and should apply to relatives sharing the no-reject/raw-map-error design, but we did not re-implement published variants, and IMEX/implicit schemes have a different failure surface we do not probe. (4) *Baseline tuning asymmetry*: Adam received a modest lr grid; the RK variants an equally modest h_0/ρ grid; the repaired method's documented h_0 fragility means a finer sweep could move results in either direction. (5) *Repaired-controller generality*: the $\sim 34\times$ result is full-batch, deterministic, and fragile to h_0; we have no evidence it survives minibatch noise, where the error estimate is variance-dominated by our own diagnosis—repairing the controller does not repair the estimator. (6) *Compute-matching granularity*: we count gradient evaluations and ignore per-step optimizer overhead, which slightly *favors* the RK methods. (7) *The implicit-regularization observation is post hoc*, emerging from inspection of result tables, and is presented accordingly. (8) *Minimum-selection mechanism is identified only by exclusion*: we do not instrument loss-surface curvature at the selected minima, so the implicit-bias mechanism is

named, not measured.

11. Conclusion

We set out to test a clean and appealing idea—that better ODE integrators make better optimizers—and found the honest answer to be *mostly no, for an instructive reason*. Under a protocol that denominates budget in gradient evaluations, a representative Bogacki–Shampine 3(2) RK–Adam loses to plain Adam on training loss in both stochastic and full-batch settings, and its "adaptive" step controller turns out to be inert: the step size saturates on the first iteration and the tolerance parameter has no effect on the trajectory at all. Diagnosing the inertness and repairing it reverses the full-batch training-loss result by $\sim 34\times$, but a fixed-step control localizes the benefit to *adaptivity*—an emergent warmup-and-growth schedule—rather than to higher-order accuracy, and the benefit does not reach test accuracy. A pre-registered temperature sweep rules out the two off-the-shelf explanations for that gap, pinning the generalization shortfall on *which* minimum the controller's trajectory selects rather than on how deep it minimizes.

The durable contributions are therefore methodological rather than a new optimizer: (1) a per-gradient-evaluation accounting discipline, with FSAL credited only where it is mathematically valid, that changes the sign of the comparison; (2) the observation that adaptive-step optimizers can ship a controller that never controls anything, together with a minimal reporting standard that would expose it; and (3) a seed-consistent implicit-regularization effect of RK gradient averaging that nonetheless never clears the tuned first-order baseline pool—a compact illustration of why optimizer claims need baseline pools rather than a single anchor. For anyone continuing the RK-optimizer program, the useful frontiers are the ones where the training-loss mechanism we isolated actually pays: deterministic, small-data, optimization-is-the-objective regimes; a variance-corrected error estimator that could make the controller meaningful under minibatch noise; and a precise account of the emergent warmup schedule as an implicit error-control policy. We release all code, logs, and results to make each of these directly checkable.

Reproducibility Statement

All experiments run on CPU with deterministic seeding. Code, result JSONs, and logs are released with the paper: `RK4Optimizer.py` (optimizer implementations), `mnist_experiment.py` (Experiment 1), `fullbatch_experiment.py` (Experiment 2 and the hyperparameter cube), `fixed_controller_experiment.py` (Experiment 3), `hmax_control_experiment.py` (the fixed-step control), and `temperature_sweep_experiment.py` (Experiment 4, with the pre-registered H1/H0 criteria and automated verdict in the module docstring). Figures are regenerated by `make_trajectory_data.py` and `make_figures.py`.

References

Bhattacharjee, A., Popov, A. A., Sarshar, A., and Sandu, A. Improving the adaptive moment estimation (ADAM) stochastic optimizer through an implicit-explicit (IMEX) time-stepping approach. *arXiv preprint arXiv:2403.13704*, 2024.

Blanes, S., Casas, F., and Sanz-Serna, J. M. Numerical integrators for the Hybrid Monte Carlo method. *SIAM Journal on Scientific Computing*, 36(4):A1556–A1580, 2014. arXiv:1405.3153.

Bogacki, P. and Shampine, L. F. A 3(2) pair of Runge–Kutta formulas. *Applied Mathematics Letters*, 2(4):321–325, 1989.

Chen, T., Fox, E., and Guestrin, C. Stochastic gradient Hamiltonian Monte Carlo. In *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 2014. arXiv:1402.4102.

Cho, S., Lee, S., Kim, D., Park, J., and Park, N. AdamNODEs: When neural ODE meets adaptive moment estimation. *arXiv preprint arXiv:2207.06066*, 2022.

Dormand, J. R. and Prince, P. J. A family of embedded Runge–Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980.

França, G., Sulam, J., Robinson, D. P., and Vidal, R. Conformal symplectic and relativistic optimization. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. arXiv:1903.04100.

França, G., Jordan, M. I., and Vidal, R. On dissipative symplectic integration with applications to gradient-based optimization. *Journal of Statistical Mechanics: Theory and Experiment*, 2021(4):043402, 2021. arXiv:2004.06840.

Hairer, E., Nørsett, S. P., and Wanner, G. *Solving Ordinary Differential Equations I: Nonstiff Problems*. Springer, 2nd edition, 1993.

Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

Li, C., Chen, C., Carlson, D., and Carin, L. Preconditioned stochastic gradient Langevin dynamics for deep neural networks. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, 2016. arXiv:1512.07666.

Neal, R. M. MCMC using Hamiltonian dynamics. In Brooks, S., Gelman, A., Jones, G. L., and Meng, X.-L. (eds.), *Handbook of Markov Chain Monte Carlo*. Chapman & Hall / CRC Press, 2011.

Nesterov, Y. A method for solving the convex programming problem with convergence rate $o(1/k^2)$. *Doklady Akademii Nauk SSSR*, 269:543–547, 1983.

Su, W., Boyd, S., and Candès, E. J. A differential equation for modeling Nesterov's accelerated gradient method: Theory and insights. *Journal of Machine Learning Research*, 17(153):1–43, 2016. arXiv:1503.01243.

Welling, M. and Teh, Y. W. Bayesian learning via stochastic gradient Langevin dynamics. In *Proceedings of the 28th International Conference on Machine Learning (ICML)*, 2011.

Wenzel, F., Roth, K., Veeling, B. S., Świątkowski, J., Tran, L., Mandt, S., Snoek, J., Salimans, T., Jenatton, R., and Nowozin, S. How good is the Bayes posterior in deep neural networks really? In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020. arXiv:2002.02405.

Zhang, J., Mokhtari, A., Sra, S., and Jadbabaie, A. Direct Runge–Kutta discretization achieves acceleration. In *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018. arXiv:1805.00521.